

BIT 2214/HBT 2305 Object Oriented Analysis and Design

Assignment 1

Name: Ngeno Kibet Caleb

Registration Number: SCT221-C004-0755/2017

Q1)

- a) Clearly, distinguish between Object Oriented Analysis and Object-Oriented Design. **[4 marks]**

Object Oriented Analysis focuses on what the system should do by modeling requirements and problem domain while Object Oriented Design focuses on how the system will do using detailed solution models and architecture.

Object Oriented Design is detailed on technical refinement of algorithms, interfaces and other system components whereas Object Oriented Analysis is detailed on the concepts of objects, classes and relationships.

- b) UML diagrams are broadly categorized into structural and behavioral diagrams. Describe the two categories; hence describe the various diagrams under each category. **[8 marks]**

Structural diagrams describe static aspects of a system like components, classes and relationships while behavioral diagrams describe dynamic aspects of a system like state, transitions and workflow.

Structural diagrams act as blueprints for a building by mapping out the fixed elements that must exist in the system and show how they connect to one another, without worrying about the timing or dynamic actions.

Structural diagrams describe the static architecture of a system - what elements exist and how they relate. They include:

- Class diagram shows classes with attributes, operations, and relationships (association, inheritance, aggregation, composition)
- Component diagram depicts physical components of a system and their dependencies
- Deployment diagram shows hardware nodes and software artifacts deployed on them
- Object diagram shows instances of classes at a specific point in time (snapshot)
- Package diagram shows grouping of model elements into packages with dependencies
- Composite Structure diagram: shows internal structure of a classifier and its interactions

Behavioral diagrams describe the dynamic behavior of a system - how elements interact and change over time. They include:

- Use Case diagram shows system functionality from the user's perspective (actors and use cases)

- Sequence diagram shows message flow between objects over time (temporal ordering)
- Collaboration/Communication diagram shows object interactions focused on structural organization of objects that send/receive messages
- State Transition diagram shows life cycle of an object with states, events, and transitions
- Activity diagram shows workflow of activities and actions with decision points and parallel flows
- Timing diagram shows timing constraints on interactions between objects

c) Explain the need for a methodology in system development. [4 marks]

A system development methodology provides guidelines to follow for completing every activity in the system development life cycle, including specific models, tools, techniques, and a development life cycle. The need for a methodology arises because:

- It ensures a structured and repeatable process, reducing the likelihood of omissions or errors
- It provides standardized documentation and communication among team members and stakeholders
- It helps manage complexity by breaking the development work into manageable phases and activities
- It improves quality, maintainability, and project control through defined deliverables and milestones
- It facilitates team coordination, resource allocation, and progress tracking
- It incorporates best practices and proven techniques, increasing the likelihood of project success
- It provides a framework for managing risks and handling changes in requirements

d) Discuss the **Agile methodology** in software development. How does the methodology support OO analysis and design? [6 marks]

Agile methodology is an iterative approach to software development that emphasizes flexibility, customer collaboration, and rapid delivery of working software. It follows the Agile Manifesto values: individuals and interactions over processes and tools; working software over comprehensive documentation; customer collaboration over contract negotiation; responding to change over following a plan.

- 1.: Agile's short iterations allow OO models (class diagrams, sequence diagrams) to be progressively refined across iterations, moving from analysis to design to code
2. Use-case driven: Agile uses user stories which align well with OO use cases; each iteration implements specific user stories/use cases

3. Embracing change: OO's encapsulation, modularity, and polymorphism make it easy to adapt to changing requirements, which is a core Agile principle
4. Frequent feedback and prototyping: OO prototypes can be quickly built and tested with users in each iteration, ensuring the system meets user needs
5. Refactoring: Agile encourages continuous improvement of the OO class structure through refactoring, leading to cleaner design
6. Test-driven development: Unit tests in OO languages verify individual classes and methods, supporting Agile's emphasis on testing
7. Self-organizing teams: OO's modularity allows team members to work on different classes/components independently

Q2)

- a) The use-case model drives the entire OOAD process. Clearly explain the implication of this statement. **[4 marks]**

The statement "the use-case model drives the entire OOAD process" means that use cases are the central thread that ties all phases of object-oriented development together. The implications are:

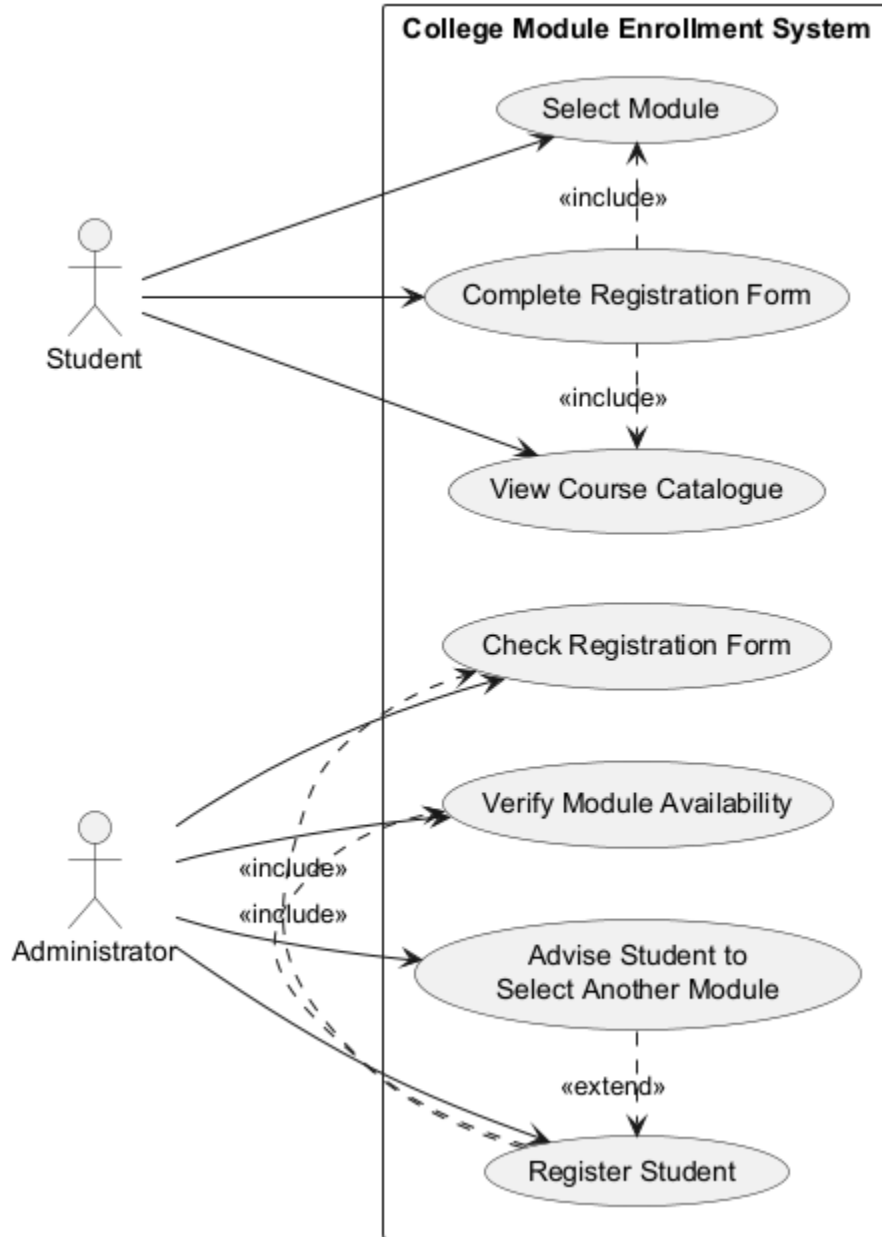
1. Use cases capture functional requirements from the user's perspective, defining what the system must do
2. Use cases guide the identification of classes and objects during analysis - nouns in use case descriptions become candidate classes
3. Use cases determine the interactions that need to be modeled in sequence and collaboration diagrams - each use case is realized through object interactions
4. Use cases drive the design of class interfaces and operations - verbs in use cases become candidate operations/methods
5. Use cases define the scope and boundary of the system, distinguishing what is inside versus outside.
6. Use cases provide a traceable thread from requirements through analysis, design, implementation, and testing - every model element can be traced back to a use case
7. Use cases serve as the basis for test case development - acceptance tests are derived from use case scenarios
8. Use cases facilitate communication between stakeholders and developers through a common, understandable notation

- b) Draw a use case diagram the following scenario **[6 marks]**

During the enrollment of modules at a college, a student completes a module registration form. The student selects units from the course catalogue and enters them in the form. Each module has a maximum capacity for students. The administrator

then checks the form and ascertains the availability of modules. If there is still space in a module, the administrator registers student on the module. If there is no space the student is advised to select another module.

Use Case Diagram for College Module Enrollment:



Via plantuml.com

Actors: Student, Administrator

Use Cases:

1. View Course Catalogue (Student)
2. Select Module (Student)
3. Complete Registration Form (Student)

4. Check Registration Form (Administrator)
5. Verify Module Availability (Administrator)
6. Register Student (Administrator)
7. Advise Student to Select Another Module (Administrator) <<extend>> Register Student

Relationships:

- Student actor is associated with View Course Catalogue, Select Module, and Complete Registration Form
- Administrator actor is associated with Check Registration Form, Verify Module Availability, Register Student, and Advise Student
- Register Student includes Check Registration Form and Verify Module Availability
- Advise Student extends Register Student (alternative flow when no space available)

The system boundary box encloses all use cases, with Student and Administrator positioned outside as external actors.

c) Show how the UML class diagram supports the following views of a system: **[6 marks]**

1. Conceptual View (Domain Perspective):

- Focuses on the problem domain concepts and their relationships
- Shows real-world classes with meaningful attributes and associations
- Ignores software implementation details like methods, visibility, and data types
- Used during analysis to understand and model the business domain
- Example: In the ambulance system, showing AmbulanceBooking, AS3, AS2, AS1 with their attributes and inheritance

2. Specification View (Interface Perspective):

- Focuses on the interfaces that classes expose to other classes
- Shows operations/methods with their signatures (parameters and return types)
- Defines what services each class provides without detailing internal implementation
- Used during design to define contracts between components
- Example: Showing processBooking(), sendToDispatcher() operations in the ambulance booking classes

3. Implementation View (Code Perspective):

- Focuses on the actual implementation details of classes
- Shows full class details including visibility markers (+, -, #), data types, method bodies

- Includes language-specific constructs and design patterns
- Used during coding and construction phases
- Example: Showing private attributes with types (int bookingID), public operations with full signatures, and class constructors

d) Develop a fully adorned conceptual class diagram for the following scenario. **[10 marks]**

There are different sorts of Ambulance Bookings, but all of them have a booking ID (we just use a number that is printed on the ambulance booking sheet) and two addresses. The first address is the pick-up address; the second is the drop-off address. Depending on the type of ambulance booking, we use different forms –AS3 are one type of ambulance booking –they are for timed appointments. AS2 are another type of ambulance booking –they are for non-emergency pick-ups. The last type of ambulance booking is an AS1 –these are a special type of AS2 –they are for emergency pick-ups and there are a few more fields on the form that we fill out. When we process the booking we have to do some things that are common to all ambulance bookings ,but AS3 's are handled in one way ,and AS2 'sand AS1 's in another .Oh,there are a few extra things we need when we process an AS1.When we 've finished processing, all bookings are sent to the dispatcher.

Fully Adorned Conceptual Class Diagram for Ambulance Bookings:

- Superclass: AmbulanceBooking (abstract)

Attributes: - bookingID: int, - pickUpAddress: String, - dropOffAddress: String, - formType: String

Operations: + processBooking(), + sendToDispatcher(): Boolean

Subclass: AS3 (Timed Appointments) -- inherits from AmbulanceBooking

Additional Attributes: - appointmentTime: Date, - appointmentDuration: int

Additional Operations: + processAS3() (overrides processBooking)

Subclass: AS2 (Non-emergency Pick-ups) -- inherits from AmbulanceBooking

Additional Attributes: (nonspecific)

Additional Operations: + processAS2() (overrides processBooking)

Subclass: AS1 (Emergency Pick-ups) -- inherits from AS2

Additional Attributes: - emergencyCode: String, - priorityLevel: int, - patientCondition: String

Additional Operations: + processAS1() (overrides processBooking, adds emergency handling)

Relationships:

- AmbulanceBooking --|> AS3 (generalization/inheritance)
- AmbulanceBooking --|> AS2 (generalization/inheritance)
- AS2 --|> AS1 (generalization/inheritance)
- AmbulanceBooking ---> Dispatcher (association: all processed bookings are sent to dispatcher)
- AmbulanceBooking ---> Form (association: each booking uses a specific form type)

Multiplicity:

- 1 AmbulanceBooking processes to 1 Dispatcher
- 1 AmbulanceBooking uses 1 Form
- 1 Form may be used by 1 AmbulanceBooking

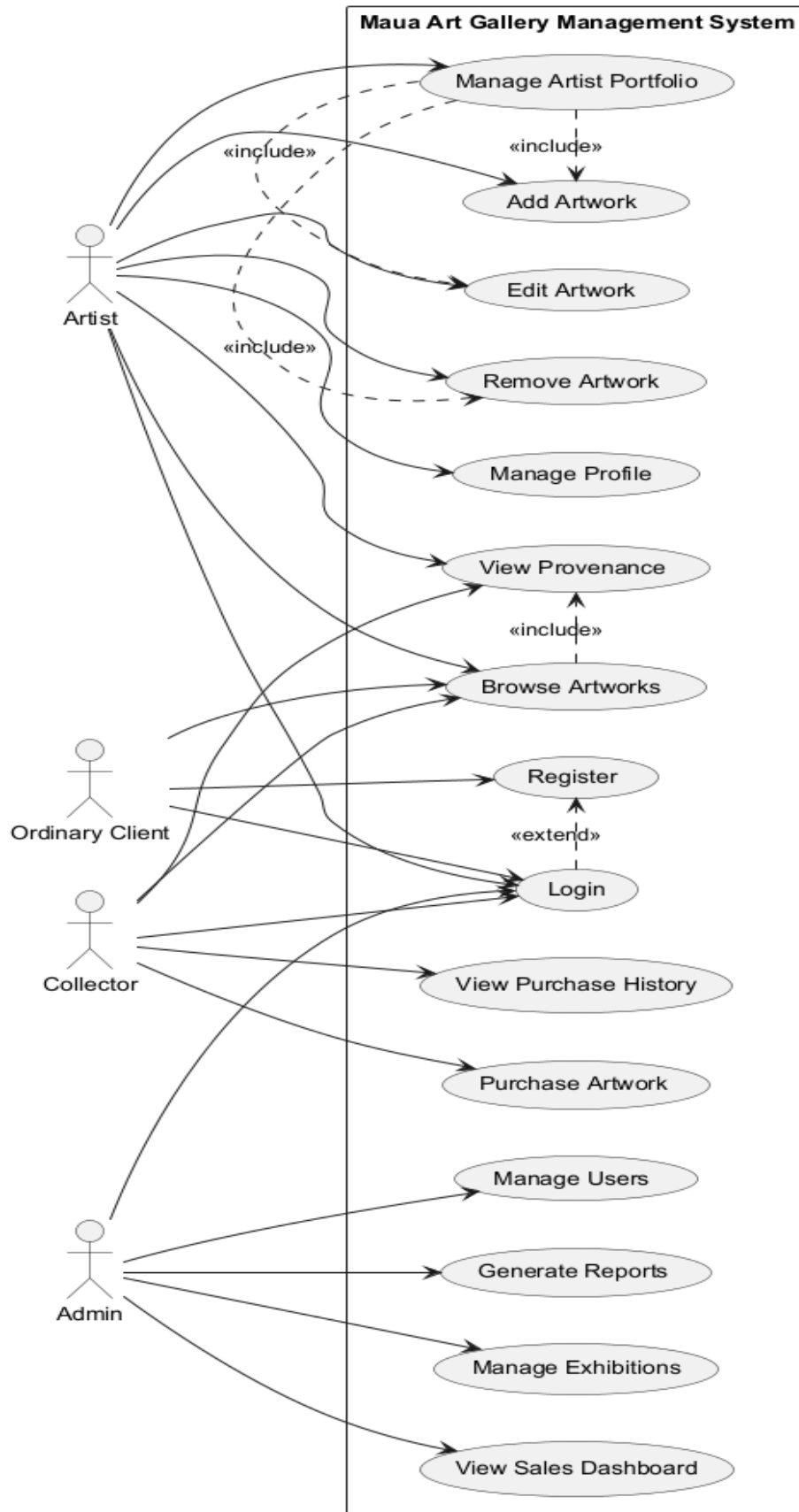
Q3)

You are developing an online management system for Maua Art Gallery located in downtown Nairobi. The firm supports registered artists to promote, exhibit and sell their works. The application should have a database to store and manage details of each artwork, including title, artist, dimensions, medium, price, and provenance. The database also stores artists, collectors and ordinary clients' information. For the artists, a detailed profile including bio, CV, exhibitions, and artwork history is stored.

Required:

- a) Develop the use case for Maua Art Gallery management system including use case diagram and written use cases. [10 marks]

Use Case Diagram for Maua Art Gallery Management System:



Implemented in plantuml.com

Actors: Artist, Collector, Ordinary Client, Administrator

Use Case:

1. Register Account (Artist, Collector, Ordinary Client)
2. Login/Logout (all actors)
3. Manage Profile (Artist - bio, CV)
4. Upload Artwork (Artist)
5. Update Artwork Details (Artist)
6. View/Browse Artworks (all actors)
7. Search for Artworks by criteria (all actors)
8. Purchase Artwork (Collector, Ordinary Client)
9. Manage Exhibition (Administrator)
10. Generate Reports (Administrator)
11. Manage User Accounts (Administrator)
12. Promote Artwork (Artist, Administrator)

Written Use Case: Purchase Artwork

- Use Case: Purchase Artwork

- Actor: Collector, Ordinary Client

- Precondition: Client is logged in, artwork is available (status = available), client has sufficient funds

- Main Flow:

1. Client browses or searches for artworks
2. System displays list of matching artworks with details (title, artist, dimensions, medium, price, provenance)
3. Client selects an artwork to view full details

4. System displays artwork details including image and provenance history
 5. Client clicks "Purchase" button
 6. System prompts for payment information
 7. Client enters payment details and confirms purchase
 8. System processes payment and updates artwork status to "Sold"
 9. System records transaction with date, amount, buyer details
 10. System generates and displays receipt
 11. System notifies artist of the sale
- Alternate Flow (Insufficient Funds):
 - 7a. Payment is declined
 - 7b. System notifies client of payment failure
 - 7c. Client may retry with alternative payment method
 - Postcondition: Artwork marked as sold, transaction recorded, artist notified

Written Use Case: Upload Artwork

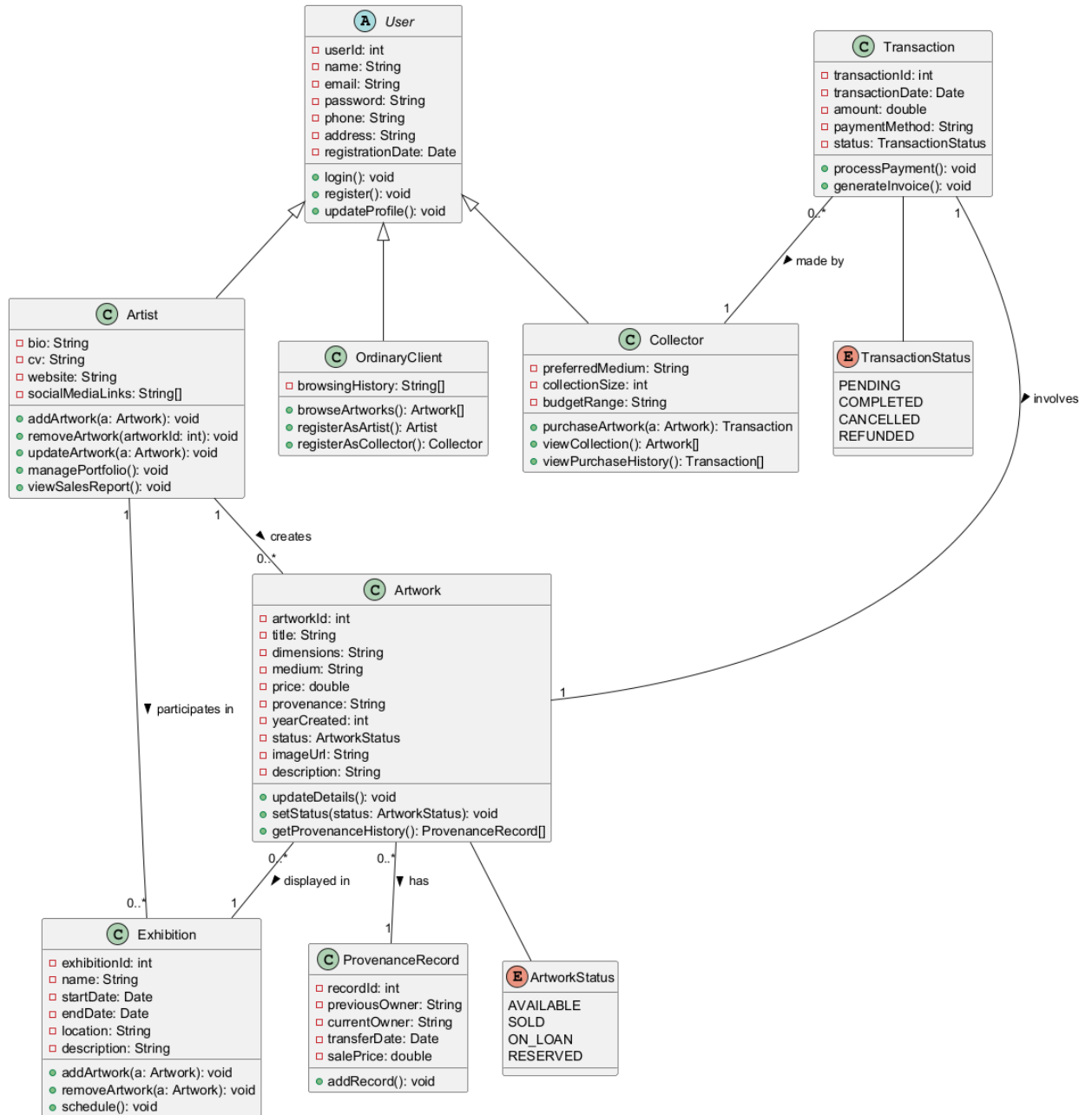
- Actor: Artist
- Precondition: Artist is logged in and has an approved profile
- Main Flow:
 1. Artist selects "Upload Artwork" option
 2. System displays artwork upload form
 3. Artist enters artwork details: title, dimensions, medium, price, provenance
 4. Artist uploads artwork images
 5. System validates and saves artwork details
 6. System sets artwork status to "Pending Approval"
 7. System confirms successful upload

- Postcondition: Artwork saved in database pending administrator approval

b) Develop a conceptual class diagram for the application.

[8 marks]

Conceptual Class Diagram for Maua Art Gallery:



Implemented in plantuml.com

Classes and Attributes:

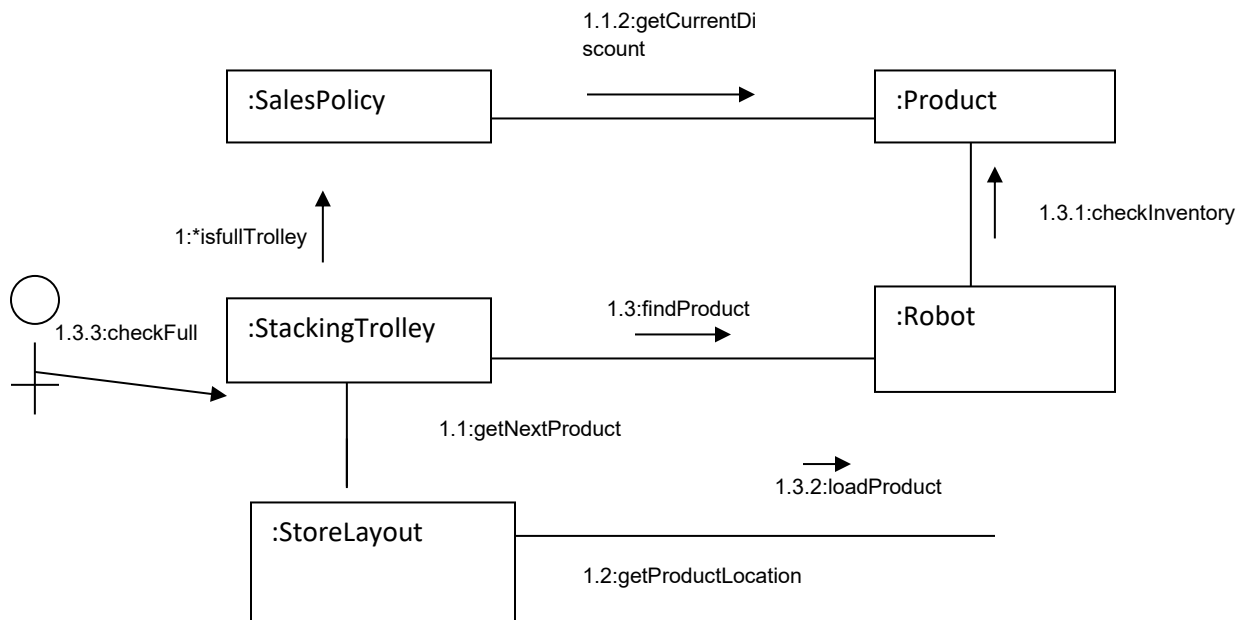
1. User (abstract)- userID: int, name: String, email: String, phone: String, password: String, registrationDate: Date
2. Artist (inherits from User)- bio: String, CV: String, artistStatement: String, website: String
3. Collector (inherits from User)- preferredGenres: String, budgetRange: String
4. OrdinaryClient (inherits from User)
5. Artwork - artworkID: int, title: String, dimensions: String, medium: String, price: double, provenance: String, status: String (available/sold/pending), imagePath: String, creationDate: Date
6. Exhibition- exhibitionID: int, title: String, description: String, startDate: Date, endDate: Date, venue: String
7. Transaction - transactionID: int, date: Date, amount: double, paymentMethod: String, status: String (completed/pending/refunded)
8. Report - reportID: int, type: String, generatedDate: Date, content: String

Relationships:

- User --|> Artist (inheritance)
- User --|> Collector (inheritance)
- User --|> OrdinaryClient (inheritance)
- Artist "1" --- "*" Artwork (one artist creates many artworks)
- Artist "*" --- "*" Exhibition (many artists participate in many exhibitions)
- Artwork "*" --- "1" Exhibition (many artworks can be in one exhibition)
- Collector "1" --- "*" Transaction (one collector makes many transactions)
- OrdinaryClient "1" --- "*" Transaction (one client makes many transactions)
- Transaction "*" --- "1" Artwork (many transactions can involve one artwork over time)
- Artist "1" --- "*" Transaction (one artist receives many transactions)

Q4)

- a) The following collaboration diagram (also known as communication diagrams in UML 2) models the interaction between a robot and a stacking trolley in the warehouse attached to a large supermarket. The stacking trolley issues commands to the robot which loads the trolley with the relevant item –ready to be taken from the warehouse to the store where the shelves can be stacked.



Convert this diagram into a sequence diagram. Explain any assumptions you have made.

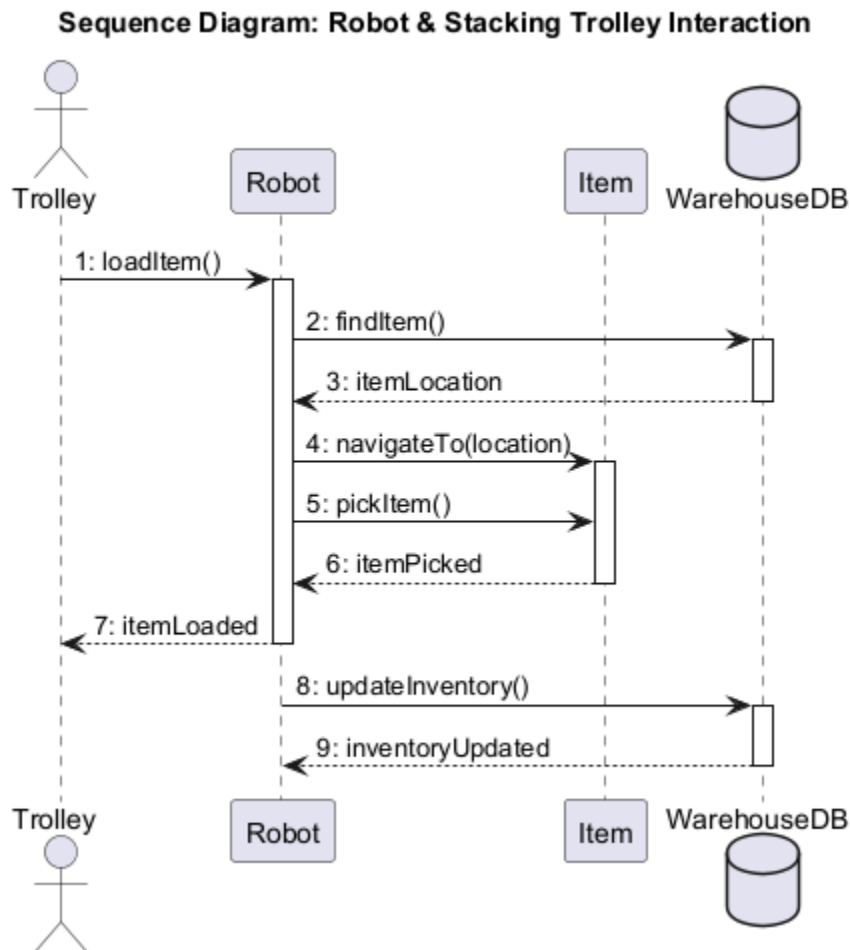
[8 marks]

Conversion of Collaboration Diagram to Sequence Diagram for Robot and Stacking Trolley:

Assumptions:

1. The collaboration diagram uses UML 1.x style numbering (1, 2, 3...) to indicate message ordering
2. The main objects involved are: Trolley (actor/initiator), Robot, Item, and WarehouseDB
3. The trolley acts as the initiator that issues commands to the robot
4. A database/system component (WarehouseDB) exists to track item locations and inventory
5. Messages are synchronous unless specified otherwise
6. The robot navigates autonomously to item locations

Sequence Diagram :



Implemented in plantuml.com

Explanation of the Sequence Diagram:

1. The Trolley initiates the process by sending a `loadItem()` message to the Robot
2. The Robot queries the WarehouseDB for the item's location using `findItem()`
3. WarehouseDB returns the location
4. The Robot navigates to the item's location
5. The Robot picks up the item
6. The Item confirms it has been picked (item is loaded onto the trolley)
7. The Robot notifies the Trolley that the item has been loaded
8. The Robot updates the inventory in WarehouseDB

9. WarehouseDB confirms the inventory update

This sequence diagram differs from the collaboration diagram in that:

- It emphasizes temporal ordering (top to bottom = time)
- Lifelines clearly show which objects exist and for how long
- Activation bars (not shown in text) would indicate when each object is active
- The flow is easier to read for complex interactions with many messages